CptS 355 Study Group — Week 2 (Sep 1 & 3)

Scribe: Claire **Attended:** Jason, Sarah, Arvind, Claire (all four)

We met Thursday around 7 in the Terrell basement, went about an hour and a half.

- Warmed up on syntax vs. semantics from Tuesday, since half of us had written basically the same sentence for both on the reading response. Arvind's version is the one that stuck: the grammar decides which strings are even sentences, the semantics decides what those sentences *do*. Same grammar, different semantics bolted on, different language.

- Then recursive descent for most of the time. Sarah pointed out that every nonterminal in the grammar becomes a function, one for one, and that the only reason it terminates is that the grammar isn't left-recursive — if `expr` started with `expr` then `parse_expr` calls itself with nothing consumed and you blow the stack. That explained why the book keeps rewriting grammars before parsing them.

- **Where we got stuck.** Three of us had written `parse_expr` basically like this, and none of us could see why `1 - 2 - 3` was evaluating to `2`:

  ```
  def parse_expr(toks):
      left = parse_term(toks)
      if peek(toks) == '-':
          toks.pop(0)
          right = parse_expr(toks)   # recursing on the whole expr
          return ('-', left, right)
      return left
  ```

  Jason drew both trees on the whiteboard and explained it to everyone and it made a lot better sense: recursing on `parse_expr` on the right builds `1 - (2 - 3)`, which is right-associative, and subtraction groups to the left. We were reading the grammar as if it only said which tokens are legal, but the recursion is also deciding the shape of the tree. Fix is a loop:

  ```
  def parse_expr(toks):
      node = parse_term(toks)
      while peek(toks) == '-':
          toks.pop(0)
          node = ('-', node, parse_term(toks))
      return node
  ```

  Now the tree grows down the left, `(1 - 2) - 3`, and you get `-4`.

- We'd never noticed with `+` because you only get punished on operators that aren't associative — Sarah's point — so `+` and `*` let you write the bug and pass all your own tests. Claire is trying `/` tonight. Arvind thinks right-associative ones like `**` are where you *do* recurse on the right, which would be a nice symmetry, but we ran out of time; he's checking before next week.

What we actually learned: associativity isn't a property of the operator symbol, it's a property of how you wrote the parsing function. All four of us thought it came from the grammar.